Two Time-Scale DRL for Service Caching and Task Offloading in Cross-Domain Marine Networks

Zhaoxiang Huang, Zhiwen Yu, *Senior Member, IEEE*, Liang Wang, *Member, IEEE*, Yingnan Zhao, *Member, IEEE*, Huan Zhou, *Senior Member, IEEE*, and Bin Guo, *Senior Member, IEEE*

***Abstract*—With increasing computational demands and limited network resources in marine environments, efficient service caching and task offloading have become critical. In such environments, Autonomous Underwater Vehicles (AUVs) rely on Uncrewed Surface Vehicles (USVs) as relays, forming a cross-domain network comprising underwater acoustic and above-water RF links. However, the heterogeneity in bandwidth, latency, and bit error rates introduces challenges for reachability analysis and delay estimation. This paper addresses the joint optimization of caching, task offloading, and resource allocation in a cross-domain marine network composed of offshore base stations, USVs, and AUVs. To tackle the inherent heterogeneity in network links and decision timescales, we formulate the problem as a two-time-scale Hierarchical Markov Decision Process (H-MDP) and propose a Two Time-Scale Deep Reinforcement Learning (T2S-DRL) approach that integrates a hybrid policy network and a lightweight structure-aware action masking mechanism. The large time-scale agent optimizes caching decisions, while the short time-scale agent focuses on offloading and resource allocation. Extensive simulations show that our approach significantly reduces task execution delay and energy consumption, validating its effectiveness.**

***Index Terms*—Marine networks, service caching, task offloading, two time-scale, deep reinforcement learning.**

I. Introduction

In recent years, the increasing demand for marine sensing has driven the deployment of various oceanic devices, such as Uncrewed Surface Vehicles (USVs) and Autonomous Underwater Vehicles (AUVs) [1], to execute computationally intensive tasks, including offshore exploration, ocean monitoring, and submarine rescue [2], [3]. These tasks require both extensive marine sensing data as input and computational services (e.g., artificial intelligence models, libraries, and algorithms) to process the data [4], [5]. For instance, in offshore exploration, AUVs equipped with underwater optical cameras capture data that must be processed using deep neural networks for target detection. To enhance computational efficiency, marine edge computing has emerged as a promising solution [6], leveraging edge caching and computation offloading to reduce computational latency and bandwidth cost [7]. More precisely, service caching refers to proactively storing task-specific service modules—such as image recognition algorithms, object detection models, or signal processing tools—on edge nodes including AUVs, USVs, and offshore base stations, so that AUV tasks requiring these services can be executed efficiently when needed [8], [9].

However, the constrained network resources in marine environments, such as limited communication and computing capabilities, pose significant challenges to designing efficient marine communication and computation strategies [10]. Unlike terrestrial communication networks, which allow mobile users to access edge infrastructures (e.g., base stations) via WiFi or 5 G, marine communication networks exhibit unique characteristics due to differences in wireless channels, network resources, and transmission media [11], [12]. Specifically, marine communication consists of both underwater and Radio Frequency (RF) transmission segments. Underwater devices, such as AUVs, cannot directly connect to terrestrial infrastructure, such as harbor base stations [13]. Instead, relay transmission, typically facilitated by USVs, is required. In this setup, AUVs and USVs communicate via acoustic channels, while USVs decode acoustic signals into RF signals for transmission to onshore base stations [10]. However, underwater acoustic communication suffers from severe packet loss and low data transmission rates [14]. Given these challenges, developing advanced marine communication and computation strategies is crucial for optimizing resource utilization in oceanic networks. Addressing these issues will significantly improve the performance and efficiency of marine edge computing systems, enabling more reliable and effective execution of marine sensing and computational tasks.

In this paper, we consider the service caching and task offloading problem in a three-tier marine network architecture, as illustrated in Fig. 1, which consists of offshore base station, USVs, and AUVs. Due to resource constraints, USVs and AUVs can only provide limited computational capabilities and cache a subset of services. Similar to existing studies, we assume that offshore base stations have sufficient capacity to cache

Received 9 April 2025; revised 28 July 2025; accepted 29 July 2025. Date of publication 1 August 2025; date of current version 3 December 2025. This work was supported in part by the National Key R&D Program of China under Grant 2024YFB4505502, in part by the National Natural Science Foundation of China under Grant 62332014, and in part by the Natural Science Foundation of Shaanxi Province for Distinguished Young Scholars under Grant 2023-JC-JQ-54. Recommended for acceptance by M. Segata. *(Corresponding author: Zhiwen Yu.)*

Zhaoxiang Huang, Liang Wang, Huan Zhou, and Bin Guo are with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: huangzx@mail.nwpu.edu.cn; liangwang@nwpu.edu.cn; hzhou117@nwpu.edu.cn; guob@nwpu.edu.cn).

Zhiwen Yu is with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China, and also with the School of Computer Science and Technology, Harbin Engineering University, Harbin 150001, China (e-mail: zhiwenyu@nwpu.edu.cn).

Yingnan Zhao is with the School of Computer Science and Technology, Harbin Engineering University, Harbin 150001, China (e-mail: zhaoyingnan@hrbeu.edu).

Digital Object Identifier 10.1109/TMC.2025.3594602

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

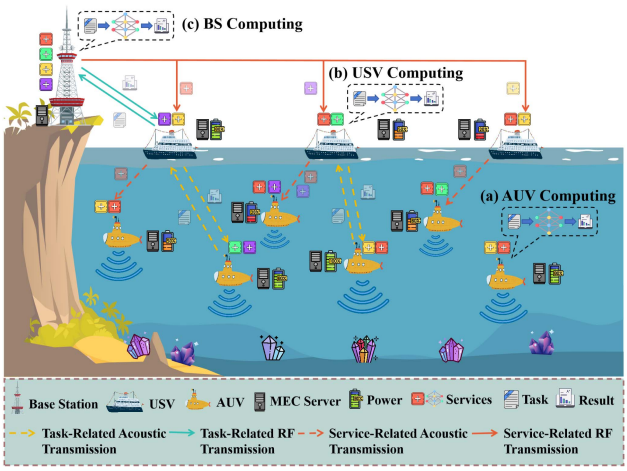


Fig. 1. An illustration of cross-domain marine network.

all services [15]. AUVs are deployed underwater to perform tasks such as resource exploration and target recognition, with the flexibility to either execute tasks locally or offload them to USVs or offshore base station. However, this system introduces several new challenges. First, the limited caching capacity of USVs and AUVs allows them to store only a subset of services, and the constrained coverage of USVs means they can only serve a portion of AUVs. Moreover, the communication between USVs and AUVs relies on acoustic channels, which suffer from limited bandwidth, high latency, and high bit error rates, further increasing the complexity of collaborative caching and service scheduling. Therefore, given the diverse nature of underwater tasks, it becomes more challenging to design complementary and efficient service caching strategies by leveraging the coverage relationships between USVs and AUVs. Second, as service duration increases, USVs and AUVs must dynamically update their local caches based on task demands. However, frequent cache updates incur high communication cost, especially in underwater environments. The cache updates for AUVs are not only constrained by the high latency of underwater acoustic communication but also increase the computational overhead and energy consumption of USVs due to the data transcoding process. Therefore, jointly considering multiple factors and determining the optimal caching update frequency presents an additional challenge. Finally, the system comprises both water-based (USVs and AUVs) and land-based (offshore base station) infrastructures, making task offloading decisions more complex due to the need for cross-domain collaboration. A key challenge of cross-domain collaboration lies in the significant differences in communication characteristics, such as bandwidth, latency, and bit error rate, between surface and underwater environments. These disparities make task reachability assessment and delay estimation more difficult. Therefore, selecting the optimal offloading destination and offloading path is crucial for minimizing task latency and optimizing energy consumption across heterogeneous devices.

Existing studies on marine task offloading mainly focus on either surface-level scenarios with RF communication [16], [17], [18], [19], [20], [21] or underwater scenarios with acoustic links [22], [23], but rarely address collaboration across domains. Some works explore integrated underwater-surface architectures [1], [24], [25], [26], yet often assume that all nodes can process tasks, overlooking the coupling between task offloading and service caching. This limits their applicability under heterogeneous constraints and dynamic service demands.

Unlike existing works, we address the service caching and task offloading problem in a cross-domain marine network architecture consisting of offshore base stations, surface-level USVs, and underwater AUVs. Our model jointly considers the cost of underwater acoustic communication, data transcoding, RF communication, and task computation, ensuring a more comprehensive optimization framework. Furthermore, recognizing the strong correlation between tasks and services, we optimize the service caching placement and cache update frequency, thereby enhancing system efficiency. Our main contributions can be summarized as follows:

- To the best of our knowledge, we are the first to investigate service caching and task offloading in cross-domain marine network architectures. The problem is formulated as the minimization of task execution cost and caching update cost under long-term computation and storage resource constraints.
- We propose a Two Time-Scale Deep Reinforcement Learning (T2S-DRL) algorithm to solve the hierarchical service caching and task offloading problem. T2S-DRL models the system as a two-time-scale Hierarchical Markov Decision Process (H-MDP), with a large time-scale agent for caching and multiple short time-scale agents for offloading and resource allocation. It incorporates a hybrid policy network to handle mixed discrete-continuous actions, along with a lightweight structure-aware action masking mechanism and a tailored reward function to improve convergence and learning efficiency.
- Extensive simulation experiments are conducted to evaluate the performance of the proposed algorithm. The performance results demonstrate that our method achieves significant improvements in convergence efficiency and leads to substantial reductions in task completion latency and energy consumption.

II. Related Work

Due to the resource constraints of Marine IoT (MIoT) devices, computation offloading has been widely regarded as an effective approach to enhancing computation efficiency in MIoT systems. Existing research primarily focuses on designing marine network computing architectures that integrate cloud servers, edge servers, uncrewed aerial vehicles (UAVs), and satellites to optimize task processing latency, device energy consumption, and overall system utility. First, Yang et al. [17] and Yang et al. [20] introduced computation offloading into maritime mobile cloud networks, leveraging the powerful processing capabilities of cloud servers to alleviate the computational limitations of marine terminals such as ships. Unlike [17], [20] further addressed the challenge of high latency caused by long-distance communication between marine terminals and remote cloud servers. To

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

mitigate this issue, they introduced edge base station servers between the cloud and marine terminals, thereby enhancing task execution efficiency. Similarly, focusing on edge-assisted marine computing architectures, Wang et al. [16] proposed an energy harvesting mechanism to address the limited access to terrestrial power grids in maritime communication networks.

UAVs have been widely applied in marine network architectures to assist marine IoT devices in computation, thereby further improving computational efficiency. Dai et al. [26] proposed a hybrid marine and aerial multiple-access edge computing solution for a system architecture composed of offshore base stations, UAVs, and USVs, aiming to enhance the service quality of maritime communication networks. In the system architecture proposed by Wang et al. [27], UAVs act as relay nodes, optimizing both transmission power and UAV trajectories. Dai et al. [11] presented a system architecture that includes AUVs and focuses on minimizing the energy consumption of marine devices, investigating UAV-assisted multi-access computation offloading in marine networks. Lyu et al. [14], You et al. [28], and Yang et al. [29] studied the computation offloading problem in an integrated Space-Air-Ground-Sea (SAGS) architecture composed of satellites, UAVs, vessels, and Base Stations (BSs). In [14], the joint task offloading and resource allocation problem was addressed, while [28] proposed a Q-learning-based approach, and [29] investigated the optimal edge server selection problem to minimize task offloading delay and energy consumption for maritime IoT users. Additionally, UAVs can also act as task requestors. In [30] and [10], UAVs offload computational tasks to USVs for processing.

Additionally, some research efforts have focused on constructing satellite-assisted marine edge computing architectures. Zhang et al. [21] and Wang et al. [1] investigated a three-tier marine edge computing system composed of Low Earth Orbit (LEO) satellites, offshore base stations, and marine wireless devices. In [21], the authors formulated an optimization problem to minimize system energy consumption under latency constraints by jointly optimizing association variables, task partitioning, transmission power, and computation resource allocation. Meanwhile, [1] considered the inherent selfishness and limited computational capabilities of marine devices, proposing an effective incentive mechanism to encourage their participation in task processing. These works primarily address coordination challenges across space-sea-ground domains. Unlike the aforementioned studies, Jiang et al. [18] and Li et al. [19] focused on the security challenges of computation offloading in a marine network consisting of High-Altitude Platforms (HAPs), malicious eavesdroppers, base stations, and USVs. Their works highlight the need to protect data confidentiality and transmission integrity in adversarial environments. These efforts collectively reflect the diversity of challenges in marine edge computing, ranging from system-level efficiency to security and trust, but few of them explore cross-domain offloading under underwater acoustic-RF hybrid links as studied in this paper.

Unlike existing computation offloading schemes in marine network environment, this work simultaneously considers the computation offloading problem within a marine network architecture comprising offshore base stations, USVs, and AUVs, leading to more complex offloading paths. Moreover, given that existing studies often overlook service caching overhead, we model service caching on a large time scale to limit cache update cost.

III. System Model and Problem Formulation

A. Network Model

As illustrated in Fig. 1, we consider a cross-domain marine network composed of a single offshore base station, denoted as b, S USVs, represented by the set $\mathcal{S} = \{1, 2, \ldots, S\}$, and A AUVs, denoted as $\mathcal{A} = \{1, 2, \ldots, A\}$. This network operates as a two-stage acoustic-RF cooperative system, where USVs act as gateways between the underwater acoustic network and the RF network, facilitating communication between the offshore base station and the AUVs operating underwater. In this system, two types of transmissions are involved: task-related and service-related. Task-related transmissions follow a bottom-up offloading process, where AUVs offload their tasks to nearby USVs via underwater acoustic communication (UAC), and if necessary, the tasks are further forwarded to the base station via RF links. In contrast, service-related transmissions occur during the content caching phase and follow a top-down process. Specifically, services are pushed from the base station to USVs through RF links and then forwarded to AUVs via UAC. To visually differentiate these two processes, Fig. 1 uses pink arrows to represent service transmissions.

In our system, the time domain is divided into T time slots, denoted as $\mathcal{T} = \{1, 2, \ldots, T\}$. Let K denote the total number of available services, represented by the set $\mathcal{K} = \{1, 2, \ldots, K\}$, where the size of each service $k \in \mathcal{K}$ is denoted as l_k. AUVs collect marine data underwater and generate computational tasks, each of which requires processing through a specific service. We represent the task generated by AUV $a \in \mathcal{A}$ as a triplet $J_a(t) = \{l_a(t), \mu_a(t), k_a(t)\}$. Here, $l_a(t)$ denotes the data size of task $J_a(t)$, $\mu_a(t)$ represents the number of CPU cycles required for the task, and $k_a(t) \in \mathcal{K}$ indicates the service required for the computational task. Due to limited computational resources and cache capacity, AUVs and USVs equipped with small-scale edge servers can only store a subset of services. We consider the limited range of underwater acoustic communication and adopt a distance-based threshold model to determine whether an AUV can offload tasks to a USV. Specifically, an AUV can establish an offloading link with a USV only if the distance between them is less than a predefined maximum communication range D_{max}. Task execution can occur at different nodes depending on service availability and computing resources. If the required service is cached locally on the AUV, the task is executed onboard to minimize delay. When the service is not available locally but can be found on a nearby USV with sufficient computing capacity, the task is offloaded to the USV. If neither condition is met, the task is transmitted to the base station, which holds all services and has abundant computational resources. The list of important notations used in this paper is shown in Table I.

B. Communication Model

1) Acoustic Communication: At any time slot t, AUV a has a certain probability of offloading its computational tasks to the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LIST OF IMPORTANT NOTATIONS USED IN THIS PAPER

Notation	Explanation
b	Offshore base station
$\mathcal{S}$	Set of USVs, $\mathcal{S} = \{1, 2, \ldots, S\}$
$\mathcal{A}$	Set of AUVs, $\mathcal{A} = \{1, 2, \ldots, A\}$
$\mathcal{T}$	Set of time slots, $\mathcal{T} = \{1, 2, \ldots, T\}$
$\mathcal{K}$	Set of services, $\mathcal{K} = \{1, 2, \ldots, K\}$
l_k	Size of service k
$J_a(t)$	Task generated by AUV a at time t
$l_a(t)$	Data size of task $J_a(t)$
$\mu_a(t)$	Required CPU cycles for task $J_a(t)$
$k_a(t)$	Service required by task $J_a(t)$
D_{max}	Maximum underwater communication range
f	Central frequency of acoustic signal
σ	Spreading factor in acoustic model
$\mathrm{G}(f)$	Absorption coefficient at frequency f
$N(f)$	Total underwater noise power density
o_1	Shipping activity factor
o_2	Wind speed
$g_{a,s}(t)$	Channel gain from AUV a to USV s at t
$W_{s,a}(t)$	Bandwidth between USV s and AUV a
p_a	Transmission power of AUV a
$W_{b,s}(t)$	RF bandwidth from base station to USV s
p_s	Transmission power of USV s
g_0	Channel gain at reference distance $d_0 = 1$ m
G_0	Antenna gain
β_1	Path loss exponent
β_2	NLoS attenuation factor
δ^2	Gaussian white noise power
$\mathcal{T}'$	Cache update time slots
$c_{s,k}(t')$	Indicator: service k cached at USV s during t'
$c_{a,k}(t')$	Indicator: service k cached at AUV a during t'
C_s	Storage capacity of USV s
C_a	Storage capacity of AUV a
$x_{s,k}(t')$	Cache update decision for USV s and service k
$x_{a,k}(t')$	Cache update decision for AUV a and service k
η	Unit energy cost for storing 1-bit of data
ϑ_s	CPU cycles for transcoding one bit at USV s
$q_a(t)$	Offloading decision vector for task $J_a(t)$
$m_a(t)$	Computing capacity of AUV a
$m_b(t)$	Computing capacity of base station

USV s that covers it via underwater acoustic communication. Based on Urick's model [31], the transmission signal attenuation model for acoustic communication is defined as

$$\mathrm{F}\left(d_{a,s}(t), f\right) = d_{a,s}(t)^{\sigma}\mathrm{G}(f)^{d_{a,s}(t)}, \tag{1}$$

where f represents the central frequency of the acoustic signal, σ is the spreading factor, $d_{a,s}(t)$ denotes the distance between the AUV a and the USV s at time t, and $\mathrm{G}(f)$ is the absorption coefficient. In the Cartesian coordinate system, we define the height of the sea surface as 0, with the positions of the AUV a and the USV s at time slot t denoted as $\ell_a(t) = (x_a(t), y_a(t), z_a(t))$ and $\ell_s(t) = (x_s(t), y_s(t), 0)$, respectively. Based on this, the distance $d_{a,s}(t)$ is given by $d_{a,s}(t) = ||\ell_a(t) - \ell_s(t)||_2$. Based on Thorp's empirical formula [31], the absorption coefficient is represented as

$$\mathrm{G}(f) = 0.11\frac{f^2}{1+f^2} + 44\frac{f^2}{4100+f^2} + 2.75e^{-4}f^2 + 0.003. \tag{2}$$

Unlike conventional white noise, underwater acoustic signals are subject to the influences of turbulence noise $N_1(f)$, shipping noise $N_2(f)$, wind-driven wave noise $N_3(f)$, and thermal noise $N_4(f)$ during their propagation in the ocean, as presented in [32]. The expression is given by

$$\begin{cases} 10\log_{10} N_1(f) = 17 - 30\log_{10} f, \\ 10\log_{10} N_2(f) = 40 + 20(o_1 - 0.5) + 26\log_{10} f \\ \qquad\qquad -60\log_{10}(f + 0.03), \\ 10\log_{10} N_3(f) = 50 + 7.5\sqrt{o_2} + 20\log_{10} f \\ \qquad\qquad -40\log_{10}(f + 0.4), \\ 10\log_{10} N_4(f) = -15 + 20\log_{10} f. \end{cases} \tag{3}$$

Here, $o_1 \in (0, 1)$ denotes the shipping activity factor, and o_2 represents the wind speed. Based on the aforementioned factors, the total noise power density can be computed as

$$N(f) = N_1(f) + N_2(f) + N_3(f) + N_4(f). \tag{4}$$

Based on the study in [6], the underwater uplink channel gain from AUV a to USV s at time slot t is calculated as

$$g_{a,s}(t) = \frac{1}{\mathrm{F}\left(d_{a,s}(t), f\right) N(f) W_{s,a}(t)}, \tag{5}$$

where $W_{s,a}(t)$ denotes the allocated channel bandwidth for acoustic communication between AUV a and USV s. Using Non-Orthogonal Multiple Access (NOMA [33] [34]), Successive Interference Cancellation (SIC) requires sorting the channels based on their respective power gains relative to USV s. It is assumed that the channel gains of the AUVs follow the order $g_{1,s}(t) > g_{2,s}(t) > \ldots > g_{A,s}(t)$. According to Shannon's theorem, the data transmission rate from AUV a to USV s at time slot t is given by

$$r_{a,s}(t) = W_{s,a}(t)\log_2\left(1 + \frac{p_a g_{a,s}(t)}{\sum_{i\in\mathcal{A}, i\neq a} p_i g_{i,s}(t) + 1}\right), \tag{6}$$

where p_a is the transmission power of AUV a. Similarly, following (6), the data transmission rate $r_{s,a}(t)$ from USV s to AUV a at time slot t can be derived.

2) Radio Frequency Communication: In this system, RF communication is used between the USV and the offshore base station, with the assumption that they communicate via Orthogonal Frequency Division Multiple Access (OFDMA [35]) to eliminate interference between different links.

At any time slot t, the position of the offshore base station is represented as $\ell_b = (x_b, y_b, z_b)$, and the distance between it and USV s can be calculated as $d_{s,b}(t) = ||\ell_s(t) - \ell_b||_2$. Considering environmental obstructions, the channel state between the USV and the base station is set to Non-Line-of-Sight (NLoS), and the corresponding data rate can be calculated using the following formula:

$$r_{s,b}(t) = W_{b,s}(t)\log_2\left(1 + \frac{p_s g_0 G_0}{\delta^2 \beta_2 (d_{s,b}(t))^{\beta_1}}\right), \tag{7}$$

where $W_{b,s}(t)$ represents the spectral bandwidth of the communication channel from the USV to the base station, p_s denotes the transmission power of the USV, g_0 is the channel power gain at 1 m, $G_0 \approx 2.2846$ [36], β_1 is the path loss exponent, β_2 is the additional attenuation factor caused by the NLoS link, and δ^2 refers to Gaussian white noise. Similarly to (7), the data transfer rate $r_{b,s}(t)$ from the base station to USV s at time slot t can be calculated.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

C. Service Caching Model

Next, we provide a detailed introduction to service caching and updating on USVs and AUVs. Given the limited caching capacity of USVs and AUVs, only a subset of services can be stored, making it necessary to determine which services to cache. Additionally, considering the high communication cost associated with frequent cache updates in the marine environment, we adopt a more practical approach—updating caching decisions every τ time slots.

To formalize this, we define the set of cache update slots as $\mathcal{T}' = \{1, 2, \ldots, T'\}$. Each caching decision made at update slot $t' \in \mathcal{T}'$ governs a fixed-length period of τ system time slots, given by:

$$t \in \{(t'-1)\cdot\tau + 1,\ (t'-1)\cdot\tau + 2,\ \ldots,\ t'\cdot\tau\}.$$

This formulation explicitly defines the mapping between caching decision epochs and the corresponding system time slots where task offloading and resource allocation decisions are made.

We define the binary cache state variables $c_{s,k}(t')$ and $c_{a,k}(t')$ to represent the service caching status of USV s and AUV a at time slot t'. Specifically,

$$c_{s,k}(t'), c_{a,k}(t') \in \{0,1\}, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}, k \in \mathcal{K}, \tag{8}$$

where $c_{s,k}(t') = 1$ indicates that service k is cached on USV s at time slot t', while $c_{s,k}(t') = 0$ means it is not cached. Similarly, $c_{a,k}(t') = 1$ means that the service k is cached on AUV a at the time slot t', whereas $c_{a,k}(t') = 0$ indicates the absence of the service.

The following constraints ensure that the total number of cached services on USVs and AUVs does not exceed their storage capacity at any given time slot:

$$\begin{aligned} &\sum_{k\in\mathcal{K}} c_{s,k}(t') l_k \le C_s, \quad \forall s \in \mathcal{S}, \\ &\sum_{k\in\mathcal{K}} c_{a,k}(t') l_k \le C_a, \quad \forall a \in \mathcal{A}, k \in \mathcal{K}, \end{aligned} \tag{9}$$

where C_s and C_a denote the total caching capacity of USV s and AUV a respectively. Additionally, as time progresses, when the storage capacity of a USV or AUV reaches its limit, a Least Recently Used (LRU) replacement policy is applied to update the cached services.

We define the cache update decision variables $x_{s,k}(t')$ and $x_{a,k}(t')$ for USV s and AUV a at time t', where

$$x_{s,k}(t'), x_{a,k}(t') \in \{0,1\}, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}, k \in \mathcal{K}. \tag{10}$$

Specifically, $x_{s,k}(t') = 1$ indicates that USV s needs to download service k from the offshore base station during time slot t'. The time overhead for cache updating is given by:

$$T_{s,k}(t') = \frac{l_k}{r_{b,s}(t')}, \quad \forall s \in \mathcal{S}, k \in \mathcal{K}. \tag{11}$$

Meanwhile, based on the cache update decision, we define the storage overhead as the energy consumption required to store service k, expressed as:

$$E_{s,k}(t') = \eta l_k, \tag{12}$$

where η represents the unit energy consumption required to store 1-bit of data.

Compared to the cache updating process of USVs, the process for AUVs is more complex because AUVs cannot communicate directly with the offshore base station. We assume that if the service AUV a needs to cache is already stored in its covering USV s, it can directly download the service from the USV. Otherwise, the base station must first transmit the service to USV s via RF communication, after which USV s will transcode the service and forward it to AUV a through underwater acoustic communication. To model the cache update decision $x_{a,k}(t')$ for AUV a, we define the cache update time overhead as:

$$\begin{aligned} T_{a,k}(t') = &\left(1 - \mathbb{I}\left(c_{s,k}(t')\right)\right)\left(\frac{l_k}{r_{b,s}(t')} + \frac{\vartheta_s l_k}{\varepsilon_{s,a}(t') m_s(t')}\right) \\ &+ \frac{l_k}{r_{s,a}(t')}, \forall a \in \mathcal{A}, k \in \mathcal{K}, \end{aligned} \tag{13}$$

where ϑ_s denotes the number of CPU cycles required to transcode one bit of service information, $\varepsilon_{s,a}(t')$ represents the percentages of computational resources allocated to AUV a on USV s, $m_s(t')$ refers to the total computation resources available on USV s at time slot t', and $\mathbb{I}(c_{s,k}(t') = 1)$ is an indicator function defined as:

$$\mathbb{I}\left(c_{s,k}(t')\right) = \begin{cases} 1, & \text{if } c_{s,k}(t') = 1, \\ 0, & \text{otherwise}, \end{cases} \tag{14}$$

which indicates whether the service is already cached in the USV. Similarly, the storage energy consumption for cache updates is given by:

$$E_{a,k}(t') = \eta l_k. \tag{15}$$

D. Task Offloading Model

Each task can either be executed locally on the AUV or offloaded to a USV or the offshore base station. A strict binary offloading policy is adopted. The offloading decision is defined by the variable $q_a(t)$, which is a one-hot vector of dimension $S+2$, where S is the number of USVs and $q_{a,i}(t)$ denotes the ith dimension. If $q_{a,i=0}(t) = 1$, task $J_a(t)$ is executed locally on the AUV; If $q_{a,i=s}(t) = 1$, the task is offloaded via underwater acoustic communication to the covering USV s for execution; If $q_{a,i=S+1}(t) = 1$, the task is first offloaded to USV s via underwater communication, and then relayed via RF communication to the offshore base station for execution after transcoding.

1) AUV Computing: First, when the AUV a has locally cached the required service for the computation of the task, and sufficient local computational resources are available, the task can be executed locally. In this case, the task delay corresponds to the local computation delay, denoted as

$$T_a^{loc}(t) = T_a^{comp}(t) = \frac{\mu_a(t)}{m_a(t)}, \tag{16}$$

where $m_a(t)$ represents the available computational resources of AUV a at time t.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

Similarly, the energy consumption for completing the task corresponds to the local computational energy, represented as

$$E_a^{loc}(t) = E_a^{comp}(t) = \phi(m_a(t))^2 \mu_a(t), \tag{17}$$

where ϕ is the correlation coefficient [37].

2) USV Computing: If AUV a is within the coverage area of USV s and USV s has cached the services required for executing task $J_a(t)$, then the task can be executed on USV s. Since USV s needs to handle tasks from multiple AUVs within its coverage area simultaneously, we use $\varepsilon_{s,a}(t) \in [0,1]$ to represent the percentage of computing resources on USV s allocated to task $J_a(t)$. The task delay when executed on USV s includes both task upload delay $T_{a,s}^{comm}(t)$ and task computation delay $T_{a,s}^{comp}(t)$. Since the computation results are relatively small, the time required to send the results to the AUV is typically negligible, denoted as

$$T_{a,s}^{USV}(t) = T_{a,s}^{comm}(t) + T_{a,s}^{comp}(t) = \frac{l_a(t)}{r_{a,s}(t)} + \frac{\mu_a(t)}{\varepsilon_{s,a}(t) m_s(t)}, \tag{18}$$

where $m_s(t)$ is the total computation resource of USV s at time slot t.

Meanwhile, the energy consumption required to complete task $J_a(t)$ consists of two parts: the communication energy consumed during task upload and the computation energy consumed by executing the task, denoted by

$$\begin{aligned} E_{a,s}^{USV}(t) &= E_{a,s}^{comm}(t) + E_{a,s}^{comp}(t) \\ &= \frac{p_a l_a(t)}{r_{a,s}(t)} + \phi \left(\varepsilon_{s,a}(t) m_s(t)\right)^2 \mu_a(t). \end{aligned} \tag{19}$$

3) BS Computing: When neither the AUV nor the USV has cached the service required for the task, the computational task $J_a(t)$ generated by AUV a can be executed at the offshore base station. During the task offloading process, the USV serves as a gateway to decode the acoustic signal and transmit it to the base station. Therefore, the task delay consists of the communication delay for uploading the task to the USV $T_{a,s}^{comm}(t)$, the task transcoding delay at the USV $T_{a,s}^{trans}(t)$, the communication delay for uploading the task from the USV to the offshore base station $T_{s,b}^{comm}(t)$, and the task computation delay $T_{a,b}^{comp}(t)$ denoted as

$$\begin{aligned} T_{a,b}^{BS}(t) &= T_{a,s}^{comm}(t) + T_{a,s}^{trans}(t) + T_{s,b}^{comm}(t) + T_{a,b}^{comp}(t) \\ &= \frac{l_a(t)}{r_{a,s}(t)} + \frac{\vartheta_s l_a(t)}{\varepsilon_{s,a}(t) m_s(t)} + \frac{l_a(t)}{r_{s,b}(t)} + \frac{\mu_a(t)}{\varepsilon_{b,a}(t) m_b(t)}, \end{aligned} \tag{20}$$

where $m_b(t)$ is the total computation resource of offshore base station at time slot t, $\varepsilon_{b,a}(t)$ represents the percentages of computational resources allocated by the offshore base station to AUV a.

Similarly, the energy consumption required to complete the computational task at the offshore base station is denoted as

$$\begin{aligned} E_{a,b}^{BS}(t) &= E_{a,s}^{comm}(t) + E_{a,s}^{trans}(t) + E_{s,b}^{comm}(t) + E_{a,b}^{comp}(t) \\ &= \frac{p_a l_a(t)}{r_{a,s}(t)} + \phi(\varepsilon_{s,a}(t) m_s(t))^2 \vartheta_s l_a(t) + \frac{p_s l_a(t)}{r_{s,b}(t)} \\ &\quad + \phi(\varepsilon_{b,a}(t) m_b(t))^2 \mu_a(t). \end{aligned} \tag{21}$$

E. Cost Model

Based on the task offloading decision variable $q_a(t)$, the time required to complete task $J_a(t)$ is represented as

$$\begin{aligned} T_a^{total}(t) &= q_{a,i=0}(t) T_a^{loc}(t) + \sum_{s \in \mathcal{S}} q_{a,i=s}(t) T_{a,s}^{USV}(t) \\ &\quad + q_{a,i=S+1}(t) T_{a,b}^{BS}(t). \end{aligned} \tag{22}$$

Similarly, the energy consumption for completing task $J_a(t)$ is represented as

$$\begin{aligned} E_a^{total}(t) &= q_{a,i=0}(t) E_a^{loc}(t) + \sum_{s \in \mathcal{S}} q_{a,i=s}(t) E_{a,s}^{USV}(t) \\ &\quad + q_{a,i=S+1}(t) E_{a,b}^{BS}(t). \end{aligned} \tag{23}$$

Therefore, we define the cost model for completing the task as the weighted sum of the time and energy consumption required to complete the task, denoted as

$$Cost^{task}(t) = \sum_{a \in \mathcal{A}} \left[\alpha T_a^{total}(t) + (1-\alpha) E_a^{total}(t)\right], \tag{24}$$

where α [1] represents the weight coefficient.

In addition, we define the cost of cache updating as the weighted sum of the cache download time and the cache storage energy consumption, represented as

$$\begin{aligned} Cost^{cache}(t') = \sum_{k \in \mathcal{K}} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} [\alpha \left(x_{s,k}(t') T_{s,k}(t') + x_{a,k}(t')\right. \\ \left. T_{a,k}(t')\right) + (1-\alpha) \left(x_{s,k}(t') E_{s,k}(t')\right. \\ \left. + x_{a,k}(t') E_{a,k}(t')\right)]. \end{aligned} \tag{25}$$

F. Problem Formulation

This paper aims to minimize the average task completion cost and cache update cost by jointly optimizing the task offloading strategy $Q = \{q_a(t) \mid a \in \mathcal{A}, t \in \mathcal{T}, \}$, cache update decision $X = \{x_{s,k}(t'), x_{a,k}(t') \mid s \in \mathcal{S}, a \in \mathcal{A}, k \in \mathcal{K}, t' \in \mathcal{T}'\}$, computational resource allocation strategy $\epsilon = \{\varepsilon_{b,s}(t), \varepsilon_{s,a}(t) \mid s \in \mathcal{S}, a \in \mathcal{A}, t \in \mathcal{T}\}$, and bandwidth resource allocation strategy $W = \{W_{b,s}(t), W_{s,a}(t) \mid s \in \mathcal{S}, t \in \mathcal{T}\}$. Therefore, the problem can be formulated as

$$\mathcal{F}: \min_{\substack{Q,X,\\ \epsilon,W}} \frac{1}{T} \sum_{t=1}^{T} Cost^{task}(t) + \beta \sum_{t'=1}^{T'} Cost^{cache}(t') \tag{26}$$

$$\text{s.t. } x_{s,k}(t') \in \{0,1\}, \quad \forall s \in \mathcal{S}, k \in \mathcal{K}, t' \in \mathcal{T}', \tag{26a}$$

$$x_{a,k}(t') \in \{0,1\}, \quad \forall a \in \mathcal{A}, k \in \mathcal{K}, t' \in \mathcal{T}', \tag{26b}$$

$$\sum_{k \in \mathcal{K}} c_{s,k}(t') l_k \le C_s, \quad \forall s \in \mathcal{S}, k \in \mathcal{K}, t' \in \mathcal{T}', \tag{26c}$$

[1] The coefficient α balances delay and energy consumption. For clarity, normalization terms are omitted, but standard normalization is applied during optimization by dividing each metric by its empirical maximum, following common practice.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

$$\sum_{k\in\mathcal{K}} c_{a,k}(t')l_k \le C_a, \quad \forall a\in\mathcal{A}, k\in\mathcal{K}, t'\in\mathcal{T}', \tag{26d}$$

$$q_{a,i}(t)\in\{0,1\}, \quad \forall i\in\{0\}\cup\mathcal{S}\cup\{S+1\}, a\in\mathcal{A}, t\in\mathcal{T}, \tag{26e}$$

$$\sum_{i\in\{0\}\cup\mathcal{S}\cup\{S+1\}} q_{a,i}(t)=1, \quad \forall a\in\mathcal{A}, t\in\mathcal{T}, \tag{26f}$$

$$\varepsilon_{b,s}(t)\in[0,1], \quad \forall s\in\mathcal{S}, t\in\mathcal{T}, \tag{26g}$$

$$\varepsilon_{s,a}(t)\in[0,1], \quad \forall s\in\mathcal{S}, a\in\mathcal{A}, t\in\mathcal{T}, \tag{26h}$$

$$\sum_{s=1}^{S}\varepsilon_{b,s}(t)\le 1, \quad \forall t\in\mathcal{T}, \tag{26i}$$

$$\sum_{a=1}^{A}\varepsilon_{s,a}(t)\le 1, \quad \forall s\in\mathcal{S}, t\in\mathcal{T}. \tag{26j}$$

Here, β represents the weight coefficient. Constraints (26a) and (26b) ensure that the cache update decisions are binary discrete variables, while Constraints (26c) and (26d) limit the number of services cached on each USV and AUV in any given time slot to not exceed their cache capacities. Constraints (26e) and (26f) impose restrictions on task offloading decisions, ensuring that each task is executed at only one location. Constraints (26g)–(26j) apply to the continuous-valued computational resource allocation variables, ensuring that the computational resources allocated to each USV and AUV do not exceed their available computational capabilities.

Theorem 1: The problem (26) is NP-hard.

Proof: We show that the formulated optimization problem $\mathcal{F}$ is NP-hard via reduction from the well-known Generalized Assignment Problem (GAP) [38]. The GAP is defined as follows: given n items and m bins, the objective is to find a minimum-cost assignment of items in which all bins do not exceed their weight budgets b_i. This can be formulated as:

$$\min \sum_{i=1}^{m}\sum_{j=1}^{n} c_{i,j}x_{i,j} \tag{27}$$

$$\text{s.t. } x_{i,j}\in\{0,1\}, \quad i=\{1,2,\ldots,m\}, \quad j=\{1,2,\ldots,n\}, \tag{27a}$$

$$\sum_{i=1}^{m} x_{i,j}=1, \quad j=\{1,2,\ldots,n\}, \tag{27b}$$

$$\sum_{j=1}^{n} r_{i,j}x_{i,j}\le b_i, \quad i=\{1,2,\ldots,m\}. \tag{27c}$$

where $c_{i,j}$ is the cost of placing item j into bin i, $r_{i,j}$ is the occupied weight, and $x_{i,j}=1$ when item j is assigned to bin i.

For the task offloading part, we treat each task generated by an AUV as an item, and the devices capable of executing the task, including AUVs, USVs, and the base station, as the m bins. Thus, the offloading decision variable $q_a(t)$ is equivalent to $x_{i,j}$, representing the problem of assigning an item to a bin. The cost associated with different task offloading strategies is referred to as $Cost^{task}(t)$, and the allocated computational resources are considered as the weight. Similarly, for the cache update part, we treat each service as an item, and USVs and AUVs as the bins. Therefore, the cache update decisions $x_{s,k}(t')$ and $x_{a,k}(t')$ are equivalent to $x_{i,j}$, and the cost of different cache update strategies is referred to as $Cost^{cache}(t')$. Consequently, Constraints (26a), (26b) and (26e) are equivalent to Constraints (27a). Constraint (26f) is equivalent to Constraint (27b). Constraints (26c), (26d) and (26g)–(26j)) are equivalent to Constraint (27c) Based on this, we can conclude that the instance of our problem is polynomial-time reducible to GAP, thus proving that the problem $\mathcal{F}$ is NP-hard. Traditional mathematical methods, such as convex optimization, are difficult to solve such optimization problems within a reasonable time frame. Therefore, in the next section, we will introduce how to apply DRL to solve this problem.

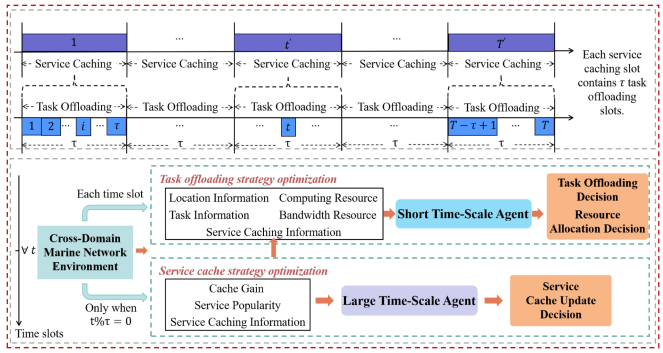


Fig. 2. Overall workflow of T2S-DRL and corresponding time-slot mapping.

IV. Algorithm Design

A. Algorithm Overview

Due to the inherent non-stationarity of the cross-domain marine network environment and the hybrid nature of our optimization problem—comprising both discrete caching and offloading decisions, as well as continuous resource allocation variables—Proximal Policy Optimization (PPO) is particularly suitable for this scenario [39]. By leveraging PPO with ratio clipping, we ensure training stability while effectively handling the mixed discrete-continuous action space. To this end, we propose a Two Time-Scale Deep Reinforcement Learning (DRL) framework based on PPO, referred to as T2S-DRL. The overall workflow of the proposed algorithm is illustrated in Fig. 2, while the architectures of the short time-scale agent and the large time-scale agent are depicted in Figs. 3 and 4, respectively.

In particular, the short time-scale agent (Fig. 3) is responsible for generating task offloading decisions and continuous resource allocation for each time slot. It integrates a discrete action network and a continuous action network to jointly address the hybrid action space. The large time-scale agent (Fig. 4), in contrast, generates caching strategies every τ time slots, adapting to long-term dynamics in content popularity and network state.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

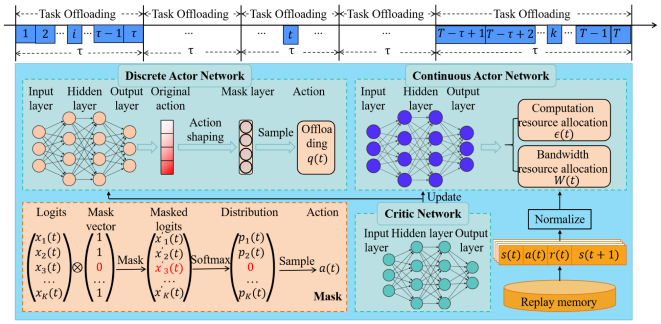


Fig. 3. Overview of the short time-scale agent in the proposed T2S-DRL framework.

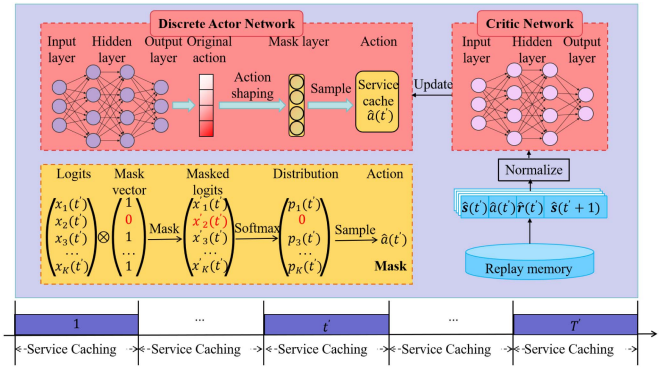


Fig. 4. Overview of the large time-scale agent in the proposed T2S-DRL framework.

Moreover, taking into account the coupling relationship between task offloading and service caching (i.e., a task can only be offloaded to a device that has cached the corresponding service), we design an illegal action masking mechanism to filter out infeasible actions, thereby enhancing the convergence performance of the algorithm. In the practical operational environment, the large time-scale model responsible for service caching updates executes periodically, while the short time-scale model, which handles task offloading and resource allocation strategies, runs at each time slot, as described in Algorithm 1. Next, we will provide a detailed explanation of the short time-scale architecture design, the large time-scale architecture design, and the action masking mechanism in the H-MDP.

B. Short Time-Scale Arichitecture Design

The short time-scale agent is responsible for learning the optimal computational task offloading strategy. Specifically, in each time slot t, it takes an offloading action $a(t)$ based on the short time-scale state $s(t)$, follows the strategy $\pi(t)$, and receives the short time-scale instantaneous reward $r(t)$. The short time-scale state, action, and reward are defined as follows:

1) Short Time-Scale State: The decision in time slot t depends on the current positions of all devices, task information, service caching information, computing resource information and bandwidth resource information, denoted as $s(t) = \{\ell(t), J(t), C(t), M(t), W(t)\}$, where

Algorithm 1: T2S-DRL Algorithm.

1: **Initialize** the cross-domain marine network environment;
2: **Load** large time-scale model, short time-scale model;
3: **for** each episode **do**
4: **# Update the local caches**;
5: **for** time slot $t = 0, 1, \ldots, T$ **do**
6: **if** $t\%\tau == 0$ **then**
7: Update the local caches of AUVs and USVs using large time-scale model;
8: **end if**
9: **# Generate offloading actions and resource allocation decisions**;
10: Use short time-scale model to determine the task offloading locations and resource allocation;
11: **end for**

$\ell(t) = \{\ell_b, \ell_s(t), \ell_a(t) | \ \forall s \in \mathcal{S}, a \in \mathcal{A}\}$, $J(t) = \{J_a(t) | \ \forall a \in \mathcal{A}\}$, $C(t) = \{c_{s,k}(t), c_{a,k}(t) | \quad \forall s \in \mathcal{S}, a \in \mathcal{A}, k \in \mathcal{K}\}$, $M(t) = \{m_b(t), m_s(t), m_a(t) | \quad \forall s \in \mathcal{S}, a \in \mathcal{A}\}$, and $W(t) = \{W_b(t), W_s(t) | \ \forall s \in \mathcal{S}\}$, Where $W_b(t)$ and $W_s(t)$ are the total bandwidth resources of the marine base station and USV s.

2) Short Time-Scale Action: The action in time slot t includes the computational task offloading decision and the resource allocation decisions for computation and bandwidth, denoted as $a(t) = \{Q(t), \epsilon(t), W(t)\}$, where $Q(t) = \{q_a(t) | \forall a \in \mathcal{A}\}$, $\epsilon(t) = \{\varepsilon_{b,a}(t), \varepsilon_{s,a}(t) \ | \ \forall s \in \mathcal{S}, a \in \mathcal{A}\}$ and $W(t) = \{W_{b,s}(t), W_{s,a}(t) \mid \forall s \in \mathcal{S}, a \in \mathcal{A}\}$.

3) Short Time-Scale Reward: Since our goal is to minimize the task completion cost, while the objective of reinforcement learning is to maximize long-term rewards, we define the reward for the short time-scale agent at time slot t as the negative of the total completion cost of all tasks in that time slot, represented as

$$r(t) = -Cost^{task}(t). \tag{28}$$

The short time-scale agent needs to generate both discrete task offloading strategies and continuous computation and bandwidth resource allocation strategies. Therefore, we design two independent actor networks that output discrete and continuous actions, with network parameters denoted as θ_d and θ_c, respectively. Similar to the Actor-Critic network architecture, this design also includes a critic network, with network parameters denoted as ϕ. To achieve this, we decompose the short time-scale strategy π into a discrete strategy π_d, which outputs discrete actions $a_d(t) = \{q_a(t) | \ \forall a \in \mathcal{A}\}$, and a continuous strategy π_c, which outputs continuous actions $a_c(t) = \{\epsilon(t), W(t)\}$. For the discrete actor network, the observed state $s(t)$ is mapped through a hidden layer to A heads (representing tasks of A AUVs). Each head then generates $S + 2$ numbers (corresponding to the AUV itself, USVs, and base station, respectively), which are passed through a softmax function to generate the probability values for selecting AUV task offloading positions, denoted as $\pi_{\theta_d}(s(t)) = [\pi_{\theta_d}(0|s(t)), \pi_{\theta_d}(1|s(t)), \ldots, \pi_{\theta_d}(S+2|s(t)),]$. For the continuous actor network, the stochastic policy π_c is generated by

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

outputting the mean and variance of the Gaussian distribution for all continuous actions.

In short time-scale network, the probability ratio $\zeta_t(\theta_d)$ is used to quantify the change in policy before and after the update, represented as

$$\zeta_t(\theta_d) = \frac{\pi_{\theta_d}(a(t)|s(t))}{\pi_{\theta_d}^{old}(a(t)|s(t))}. \tag{29}$$

To stabilize and smooth the learning process, Generalized Advantage Estimation (GAE) is integrated into the network, denoted as

$$A_t = \rho_t + (\gamma\lambda)\rho_{t+1} + (\gamma\lambda)^2\rho_{t+2} + \ldots, \tag{30}$$

where $\lambda \in [0,1]$ is the weighting parameter and $\gamma \in [0,1]$ is reward discount factor. A clipping function is used to limit the extent of policy updates, represented as

$$\text{clip}\left(\zeta_t(\theta_d), 1-\epsilon, 1+\epsilon\right) = \begin{cases} 1-\epsilon, & \text{if } \zeta_t(\theta_d) \le 1-\epsilon, \\ 1+\epsilon, & \text{if } \zeta_t(\theta_d) \ge 1+\epsilon, \\ \zeta_t(\theta_d), & \text{otherwise.} \end{cases} \tag{31}$$

Therefore, the objective for the discrete actor network can be expressed as

$$L_\pi^{clip}(\theta_d) = \mathbb{E}_t\left[\min\left(\zeta_t(\theta_d)A_t, \text{clip}\left(\zeta_t(\theta_d), 1-\epsilon, 1+\epsilon\right) A_t\right)\right]. \tag{32}$$

The parameter update process of continuous actor network is similar to that of discrete actor network.

The critic network is used to generate the state value function $V_\phi(s(t))$, which quantifies the expected return from a given state $s(t)$, represented as

$$V_\phi(s(t)) = \mathbb{E}_t\left[\sum_{k=0}^{\infty}\gamma^k r(t+k)|s(t)\right]. \tag{33}$$

The objective of the critic network is denoted as

$$L_\pi(\phi) = \mathbb{E}_t[(V_\phi(s(t)) - V_t^{target})^2], \tag{34}$$

where V_t^{target} is the target value obtained using GAE. The detailed training process of the algorithm is presented in Algorithm 2.

Algorithm 2: Short Time-Scale Algorithm.

Initialize the actor networks and critic networks with parameters θ_d, θ_c, ϕ replay memory$\mathcal{M}_s$;
for each episode **do**
  Initialize environment;
  for time slot $t = 0, 1, \ldots, T$ **do**
    Obtain the services cache information;
    Observe the current state $s(t)$;
    Select action $a_d^{orign}(t) \sim \pi_d$ and $a_c^{orign}(t) \sim \pi_c$;
    Obtain the correct action $a_d(t)$ and $a_c(t)$ through the action masking mechanism;
    Execute the action $a_d(t)$ and $a_c(t)$ to obtain the reward $r(t)$;
    Observation the next state $s(t+1)$;
    Save $(s(t), a(t), r(t), s(t+1), a_d(t), a_c(t))$ to $\mathcal{M}_s$;
  end for
  Shuffle data and create mini-batches for updates;
  Adam optimizer updates θ_d, θ_c, and ϕ by maximize (32) and minimize (34);
  Update $\theta_d^{old} \leftarrow \theta_d$, $\theta_c^{old} \leftarrow \theta_c$ and $\phi^{old} \leftarrow \phi$;
  Clear the replay memory$\mathcal{M}_s$;
end for

C. Large Time-Scale Arichitecture Design

The large time-scale agent is responsible for learning the optimal cache update strategy. Specifically, in each cache update slot t', it takes a cache update action $\hat{a}(t')$ based on the large time-scale state $\hat{s}(t')$, follows the policy $\hat{\pi}$, and receives the large time-scale instantaneous reward $\hat{r}(t')$. The large time-scale state, action, and reward are defined as follows:

1) Large Time-Scale State: Since the service caching strategy heavily relies on the offloading decisions at the short time scale, the state space is composed of the current service cache configurations of each USV and AUV, the accumulated cache gain for each cache, and the popularity of each service on the USV and AUV, denoted as $\hat{s}(t') = \{C(t'), g(t'), p(t')\}$. Here, $C(t') = \{c_{s,k}(t'), c_{a,k}(t')| \ \forall s \in \mathcal{S}, a \in \mathcal{A}, k \in \mathcal{K}\}$, $g(t') = \{g_k(t')| \ \forall k \in \mathcal{K}\}$, and $p(t') = \{p_k(t')| \ \forall k \in \mathcal{K}\}$. The caching gain of each service reflects the cost reduction achieved by executing tasks requiring service k on USVs or AUVs, compared to executing them at the base station. The cost of executing all tasks requiring service k at the base station at time t is defined as

$$\text{Cost}^{\text{BS}}(t,k) = \sum_{a\in\mathcal{A}}\mathbb{I}(k_a(t)=k)\left[\alpha T_{a,b}^{\text{BS}}(t) + (1-\alpha)E_{a,b}^{\text{BS}}(t)\right], \tag{35}$$

where $\mathbb{I}(\cdot)$ is the indicator function, which returns 1 if the condition holds and 0 otherwise. Similarly, we define $\text{Cost}^{\text{USV}}(t,k)$ and $\text{Cost}^{\text{AUV}}(t,k)$ as the cost of executing the tasks requiring service k on USVs and AUVs, respectively. Based on the above, the caching gain of service k is expressed as

$$g_k(t') = \sum_{t\in t'}\left[\text{Cost}^{\text{BS}}(t,k) - \max\left(\text{Cost}^{\text{USV}}(t,k), \text{Cost}^{\text{AUV}}(t,k)\right)\right]. \tag{36}$$

The service popularity is denoted as

$$p_k(t') = \sum_{t\in t'}\frac{|\{a|\forall k_a(t)=k\}|}{\tau A}. \tag{37}$$

2) Large Time-Scale Action: The action is the service cache update decision for each USV and AUV, denoted as $\hat{a}(t') = \{x_{s,k}(t'), x_{a,k}(t')| \ \forall s \in \mathcal{S}, a\mathcal{A}, k \in \mathcal{K}\}$.

3) Large Time-Scale Reward: Since the large time-scale agent optimizes the caching cost based on the tasks within each cache update interval, while also considering the coupling relationship between tasks and caching as well as its inherent latency compared to the short time-scale agent, we define the reward by accumulating the task cost output by the short time-scale agent

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Large Time-Scale Algorithm.

Initialize the weights and biases of $\hat{\pi}_{\theta'_d}(\hat{a}(t')|\hat{s}(t'))$ and $V_{\phi'}(\hat{s}(t'))$, the replay memory $\hat{\mathcal{M}}$;
Collect experiences with policy $\pi^{old}_{\theta'_d}(\hat{a}(t')|\hat{s}(t'))$;
for each episode **do**
  for time slot $t' = 0, 1, \ldots, T'$ **do**
    Observe the current state $\hat{s}(t')$;
    Implemet the policy $\pi^{old}_{\theta'_d}(\hat{a}(t')|\hat{s}(t'))$ in cross-domain marine environment to obtain action;
    Execute the action $\hat{a}(t')$ to obtain the reward $\hat{r}(t')$;
    Observation the next state $\hat{s}(t'+1)$);
    Save $(\hat{s}(t'), \hat{a}(t'), \hat{r}(t'), \hat{s}(t'+1)$ to $\hat{\mathcal{M}}$;
  end for
  # Optimize the neural networks;
  for epoch $= 1$ to K **do**
    Shuffle data and create mini-batches for updates;
    Compute GAE $\tilde{A}$ according (30);
    Adam optimizer updates the parameters by maximize (32) and minimize (34);
  end for
  Update $\theta_d^{old'} \leftarrow \theta'_d$ and $\phi^{old'} \leftarrow \phi'$;
  Clear the replay memory $\hat{\mathcal{M}}$;
end for

over the cache update interval. Thus, the reward is defined as

$$\hat{r}(t') = \sum_{t \in t'} r(t) - Cost^{cache}(t'). \tag{38}$$

Since the large time-scale agent is responsible for generating discrete cache update decisions, our proposed architecture includes a discrete actor network and a critic network, with parameters denoted as θ'_d and ϕ', respectively. For the discrete actor network, the observed state $\hat{s}(t')$ is mapped through a hidden layer to $S + A$ heads (representing USVs and AUVs). Each head then generates K numbers (representing the number of services), which are passed through a softmax function to produce the probability values for the cache update actions, denoted as $\hat{\pi}_{\theta'_d}(\hat{s}(t')) = [\hat{\pi}_{\theta'_d}(0|\hat{s}(t')), \hat{\pi}_{\theta'_d}(1|\hat{s}(t')), \ldots, \hat{\pi}_{\theta'_d}(K|\hat{s}(t')),]$. The action taken at time slot t' can then be sampled from this distribution.

Similarly, based on (32), the objective function for the discrete actor network of the long-time-scale agent can be defined, and based on (34), the objective function for the critic network can be formulated. The parameter update process follows the same procedure as that of the short-time-scale agent, with the detailed process provided in Algorithm 3.

D. Action Masking Mechanism Design

In addition, since the task offloading location depends on the caching strategy and tasks cannot be offloaded to devices that do not have the corresponding service cached, traditional DRL methods typically use penalty terms to constrain invalid actions. However, this approach can reduce algorithm convergence performance and cannot completely avoid invalid actions. To address this, we design an action masking mechanism to filter out invalid actions.

To begin with, we define a vector $X_a(t) = [x_{a,1}(t), x_{a,2}(t), \ldots, x_{a,S+2}(t)] \in \mathbb{R}^{S+2}$ to represent the initial logits corresponding to the possible task offloading locations for a specific task $J_a(t)$. Based on this, the initial logits for all tasks at time slot t, as output by the discrete actor network, can be concatenated into a matrix $\mathrm{X}(t) \in \mathbb{R}^{A\times(S+2)}$, where A is the total number of tasks, $S+2$ denotes the total number of candidate nodes for task offloading, including the S USVs, the AUV itself, and the base station. This matrix is denoted as:

$$\mathrm{X}(t) = \begin{bmatrix} x_{1,1}(t), x_{1,2}(t), \ldots, x_{1,S+2}(t) \\ x_{2,1}(t), x_{2,2}(t), \ldots, x_{2,S+2}(t) \\ \vdots \\ x_{A,1}(t), x_{A,2}(t), \ldots, x_{A,S+2}(t) \end{bmatrix}_{\in \mathbb{R}^{A\times(S+2)}}. \tag{39}$$

To ensure that tasks are only offloaded to nodes that have cached the required service, we introduce a lightweight binary mask vector $V_a(t) = [v_{a,1}(t), v_{a,2}(t), \ldots, v_{a,S+2}(t)] \in \mathbb{R}^{S+2}$ to represent whether the AUV, USVs, and base station have cached the service required for task $J_a(t)$. Specifically,

$$v_{a,i}(t) = \begin{cases} 1, & \text{if } c_{i,k}(t) = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{40}$$

Accordingly, a binary mask matrix $\mathrm{V}(t) \in \mathbb{R}^{A\times(S+2)}$ is constructed, where each row represents the service availability for a specific task across all candidate nodes. Specifically, the matrix can be denoted as

$$\mathrm{V}(t) = \begin{bmatrix} v_{1,1}(t), v_{1,2}(t), \ldots, v_{1,S+2}(t) \\ v_{2,1}(t), v_{2,2}(t), \ldots, v_{2,S+2}(t) \\ \vdots \\ v_{A,1}(t), v_{A,2}(t), \ldots, v_{A,S+2}(t) \end{bmatrix}_{\in \mathbb{R}^{A\times(S+2)}}. \tag{41}$$

With this, we generate the masked logits matrix $\mathrm{X}'(t) \in \mathbb{R}^{A\times(S+2)}$, formulated as:

$$\mathrm{X}'(t) = \mathrm{X}(t) \odot \mathrm{V}(t), \tag{42}$$

where $\odot$ denotes the element-wise Hadamard product. For each task $J_a(t)$, the corresponding masked logits vector $X'_a(t) \in \mathbb{R}^{S+2}$ ensures that only valid offloading options (i.e., nodes that have cached the required service) are considered during the decision process. After obtaining $\mathrm{X}'(t)$, a softmax operation is applied to produce a probability distribution over offloading locations, enabling probabilistic sampling of the offloading decision, denoted as

$$\mathrm{P}(t) = softmax(\mathrm{X}'(t)). \tag{43}$$

Based on this final probability distribution, sampling the offloading action enables the masking of invalid actions.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS AND EXPERIMENTAL SETTINGS

Parameters	Value
The spreading factor, σ	1.5
The central frequency of acoustic signal, f	1 kHz
The shipping activity factor, o_1	0.5
Wind speed, o_2	5 m/s
The transmission power of AUVs, p_a	0.1 W
The total transmission bandwidth of BS	4 MHz
The channel power gain at l m, g_0	1.42×10^{-4}
The transmission power of USVs, p_s	0.5 W
Path loss exponent, β_1	3
The number of CPU cycles for processing one bit of acoustic signal, ϑ_s	10^4 cycles
Gaussian white noise, δ^2	-100 dBm
The transmission power of BS, p_b	10 W
Additional attenuation factor caused by NLoS link, β_2	0.01
Energy correlation coefficient, ϕ	5×10^{-27}
Weight factor, α	0.5
Balance factor, β	0.001
The unit energy cost for storage, η	0.0001 J

V. PERFORMANCE EVALUATION

A. Evaluation Setup

We consider a cross-domain marine network consisting of one offshore base station and three USVs, which provides computation and caching services for $A = \{5, 10, 15, 20, 25, 30\}$ AUVs. These AUVs are randomly distributed within a 500×500 m^2 rectangular area, while AUVs operate at a fixed underwater depth of 50 m. This system can provide $K = 10$ different types of services. We assume a total number of time slots $T = 500$, the caching is updated every $\tau = \{5, 10, 15, 20, 25, 30\}$ time slots. At each time slot t, AUVs generate computation tasks following a Zipf distribution, where the input data size ranges from [100,500]MB, and the computational demand varies between $[10^8, 10^9]$ CPU cycles. Additionally, each USV and AUV has computational capability of [10, 20] GHz, and a maximum communication coverage radius of 300 m. These USVs and AUVs are evenly distributed within the target area. Table II summarizes all simulation parameters and experimental settings. All experiments were carried out on a Ubuntu 20.04 server equipped with 8 NVIDIA GeForce RTX 3090 graphic cards and an Intel(R) Xeon(R) Silver 4210R CPU @2.40 GHz.

For the proposed T2S-DRL model, both the actor and critic networks consist of two fully connected hidden layers, each with 64 neurons. Neural network parameters are initialized using orthogonal initialization, and the Generalized Advantage Estimation (GAE) method is adopted to compute the policy gradient. The learning rate for both the actor and critic networks is set to 5×10^{-4}. The reward discount factor is set to 0.99, the GAE parameter to 0.95, and the PPO clipping parameter to 0.2. The replay buffer size is set to 2048, and the batch size is 64.

We adopt the following baseline methods for comparison:

- *LRSCA [40]:* An iterative algorithm based on linear relaxation and successive convex approximation is employed to solve for the optimal service caching and computing offloading scheme.
- *SRMA [41]:* Based on Dinkelbach's method and convex optimization theory, a four-stage alternating iterative algorithm for service ratio maximization is proposed to solve the problem of task offloading and service cache placement.
- *IDCCO [42]:* An intelligent data caching and computing offloading algorithm, and a distributed training mechanism based on federated learning is adopted.
- *JSC2O [43]:* A hierarchical deep reinforcement learning algorithm, where multiple edge agents and a cloud agent collaboratively determine the caching-action and offloading-action, respectively.

B. Convergence Performance

First, we validate the impact of the proposed action masking mechanism on the convergence of both short and large time-scale algorithms. As shown in Figs. 5(a) and 6(a), in the absence of an action masking mechanism, agents explore a large number of invalid actions, making it difficult for the algorithm to converge. In contrast, agents equipped with the action masking mechanism effectively filter out illegal actions, allowing the algorithm to converge to the optimal solution after approximately 1000 training episodes. This significantly enhances convergence performance.

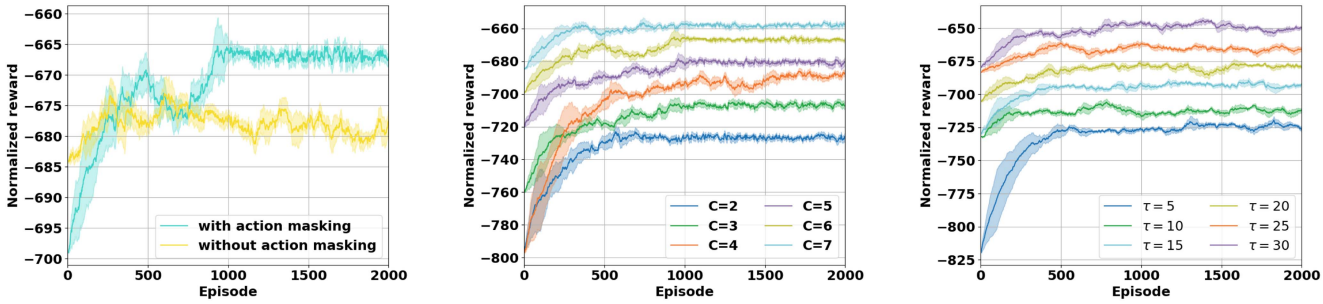
Second, we investigate the effect of the number of cache services deployed on USVs and AUVs on algorithm convergence, as illustrated in Figs. 5(b) and 6(b). The results indicate that as the number of cached services increases, more computational tasks tend to be executed locally or offloaded to nearby USVs as a priority. This reduces task transmission delay and energy consumption, leading to lower execution cost and, consequently, higher rewards.

Finally, we examine the impact of cache update frequency on algorithm convergence, as shown in Figs. 5(c) and 6(c). The experimental results reveal that as the cache update frequency increases from 5 to 30, the reward progressively increases while both task execution and cache update cost decrease. This is because a lower update frequency reduces the number of services that need to be transmitted per episode, thereby lowering caching cost. However, infrequent updates lead to a higher proportion of computational tasks being offloaded to more expensive base station, increasing overall execution cost.

C. Effects of the Weight Coefficient

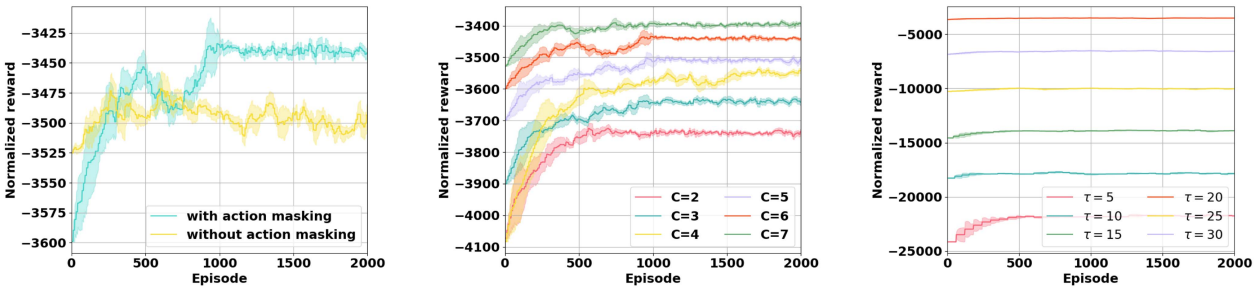
Under the condition of a cache size of 2 and a cache update frequency of 5, we evaluated the impact of the weighting factor α on average task delay and energy consumption, as shown in Fig. 7. It can be observed that as α increases, the average task delay exhibits a decreasing trend, while energy consumption shows an increasing trend. This indicates that the algorithm tends to trade off energy consumption for lower latency in the offloading decisions. When $\alpha = 0.75$, the average task delay is reduced by approximately 25% compared to that when $\alpha = 0.5$, while the energy consumption increases by around 18%.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.



(a) Action masking mechanism (b) Cache number (c) Cache update frequency

Fig. 5. Short time-scale convergence: (a) action masking mechanism, (b) cache number and (c) cache update frequency.



(a) Action masking mechanism (b) Cache number (c) Cache update frequency

Fig. 6. Large time-scale convergence: (a) action masking mechanism, (b) cache number and (c) cache update frequency.

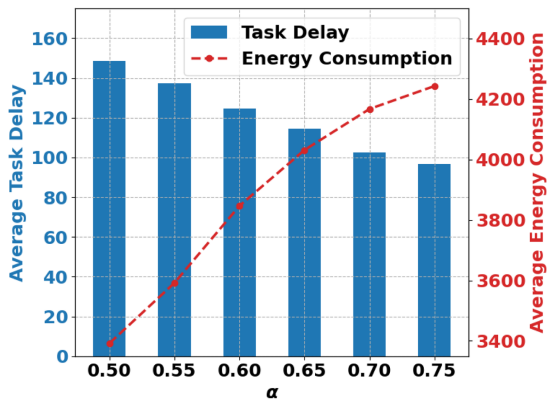


Fig. 7. Impact of weight parameter α.

D. Effects of Different Cache Numbers

Fig. 8 illustrates the impact of the number of caching services on USVs and AUVs on task execution delay, average energy consumption, and cache hit rate. As shown in Fig. 8(a), increasing the number of cached services allows more computational tasks to be executed locally or offloaded to nearby USVs, thereby reducing execution delay. Specifically, when the number of cached services increases from 2 to 7, the average task execution delay decreases by 29%. Similarly, this reduces the number of tasks that require decoding and transmission to the offshore base station, leading to lower energy consumption.

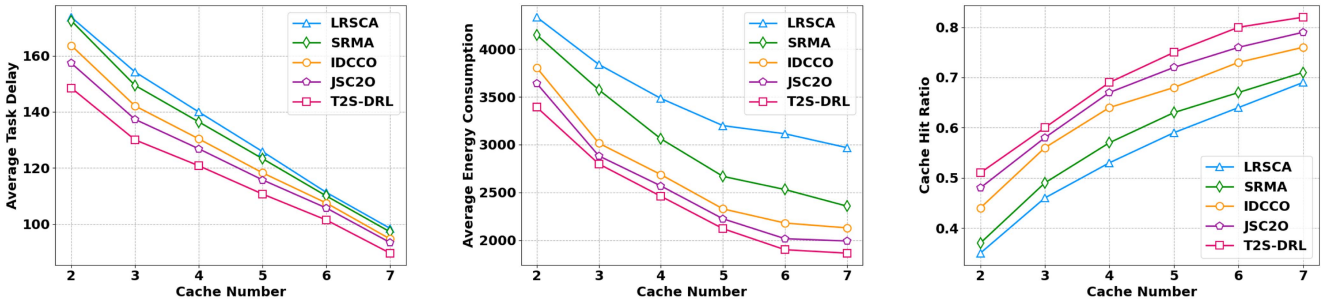
Fig. 8(b) further demonstrates that our proposed method achieves the lowest energy consumption compared to the LRSCA, SRMA, IDCCO, and JSC2O baselines, reducing energy consumption by 37%, 21%, 12%, and 6%, respectively. Additionally, as the number of cached services increases, the energy consumption decreases significantly. When the number of cached services reaches 7, our method reduces energy consumption by 45% compared to the scenario where only 2 services are cached. This improvement indicates that caching more services on USVs and AUVs minimizes the need for task offloading to the base station, thereby significantly reducing overall energy consumption.

In Fig. 8(c), we compare the cache hit ratios of different methods. The results show that our method consistently achieves the highest cache hit ratio. Compared to the best baseline approach, JSC2O, our method improves the cache hit ratio by 4%. Meanwhile, as the number of cached services increases, the cache hit ratio of our proposed method gradually improves, which is consistent with the conclusions drawn from Fig. 8(a) and (b).

E. Effects of Different Cache Update Frequencies

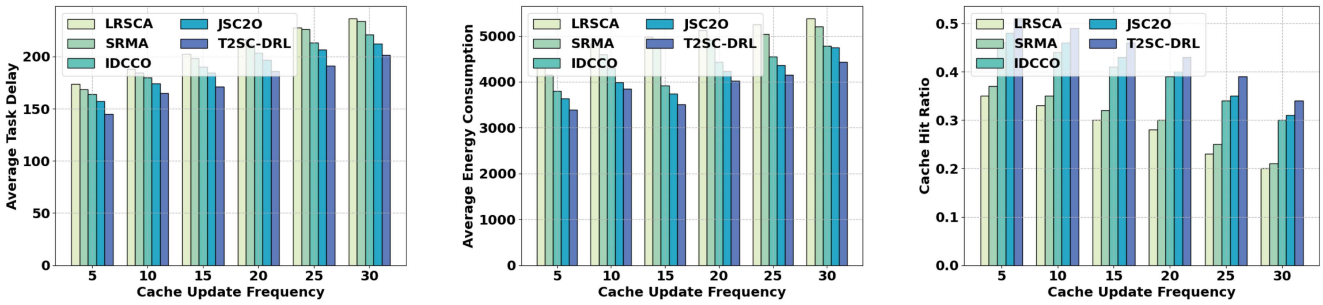
Besides the number of caching services, the cache update frequency also affects task execution delay, energy consumption, and cache hit ratio. In Fig. 9(a), we evaluate the average task execution delay under different cache update frequencies across various methods. The results indicate that our proposed method

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.



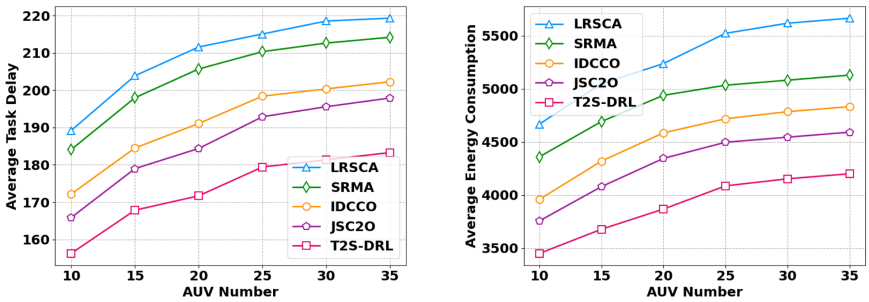
(a) Average task delay (b) Average energy consumption (c) Cache hit ratio

Fig. 8. Impact of cache number on (a) average task delay, (b) average energy consumption, and (c) cache hit ratio.



(a) Average task delay (b) Average energy consumption (c) Cache hit ratio

Fig. 9. Impact of cache update frequency on (a) average task delay, (b) average energy consumption, and (c) cache hit ratio.



(a) Average task delay (b) Average energy consumption

Fig. 10. Impact of AUV number on (a) average task delay and (b) average energy consumption.

consistently maintains the best performance. Although increasing the cache update interval slightly increases task execution delay, our method still achieves approximately 5% lower delay compared to other approaches.

Fig. 9(b) presents the impact of cache update frequency on energy consumption. As the cache update interval increases, all five algorithms exhibit a relatively slow upward trend in energy consumption. However, our method consistently maintains the lowest energy consumption, demonstrating its effectiveness in significantly reducing the number of tasks offloaded to offshore base stations.

Fig. 9(c) illustrates the effect of cache update frequency on the cache hit ratio. When the cache update interval increases from 5 to 30, the number of local cache updates per USV and AUV within a single episode decreases from 400 to approximately 66. This reduction limits the ability to execute computational tasks on USVs and AUVs, leading to a decline in the cache hit ratio. Nevertheless, compared to other baseline methods, our approach still achieves the highest cache hit ratio. When the cache update interval is set to 30, our proposed method achieves a 10% higher cache hit ratio compared to the optimal baseline method, JSC2O.

F. *Effects of Different AUV Numbers*

In Fig. 10, we evaluate the impact of the number of AUVs on task execution delay and average energy consumption. As shown in Fig. 10(a), when the number of AUVs increases, the total number of tasks generated per time slot also increases, leading to a reduction in the resources allocated to each individual

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

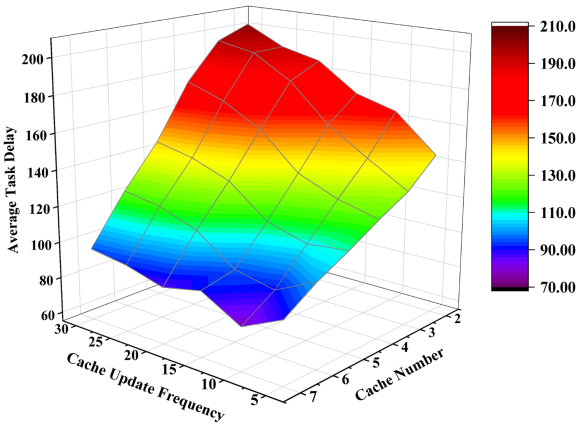


Fig. 11. The joint effect of cache number and update frequency on task delay.

task. Consequently, the average task execution delay gradually increases across all methods. However, our proposed method consistently achieves the lowest delay, reducing the delay by up to 7% compared to the optimal baseline method, JSC2O.

Similarly, Fig. 10(b) presents the impact of the number of AUVs on the average energy consumption of different methods. Consistent with the findings in Fig. 10(a), as the number of AUVs increases, more tasks are offloaded to distant execution nodes, resulting in higher transmission and decoding energy consumption. Nevertheless, our method maintains the lowest energy consumption across all scenarios, demonstrating its superiority in minimizing energy overhead.

G. Joint Impact of Cache Size and Update Frequency

In Fig. 11, we evaluate the combined impact of cache quantity and cache update frequency on task completion delay. It can be observed that when the number of cached services is small (e.g., 2) and the cache update interval is set to 30 time slots, a significant number of tasks experience high delays. This is because the USVs and AUVs lack the necessary cached services and cannot update their local caches in time, forcing task offloading to distant offshore base stations, which substantially increases delay. However, as the number of cached services increases and the cache update interval decreases, the task execution delay exhibits a downward trend. Moreover, when the number of cached services reaches 4 or more, the sensitivity of task execution delay to cache update frequency is noticeably reduced.

VI. Conclusion

In this paper, we have investigated the service caching and computation offloading problem in a cross-domain marine network architecture. The objective is to minimize task execution delay and energy consumption, by jointly optimizing service cache placement, task offloading decisions, and resource allocation strategies. We model the problem as a Hierarchical Markov Decision Process (H-MDP), where service caching and task offloading decisions are made at different time scales. To solve this problem efficiently, we propose a Two Time-Scale Deep Reinforcement Learning algorithm (T2S-DRL), where a large time-scale agent periodically updates caching strategies, and a short time-scale agent optimizes task offloading and resource allocation. Additionally, we introduce an action-masking mechanism to improve learning efficiency by eliminating infeasible actions. Extensive experiments demonstrate the effectiveness of the proposed approach, showing significant improvements in task delay, energy consumption, and cache hit ratio compared to existing methods.

In future work, we plan to investigate the challenges of service caching and task offloading under unstable maritime network conditions. The proposed framework relies on offloading computation tasks and downloading service caches, both of which assume stable and reliable communication links among AUVs, USVs, and offshore base stations. However, in real-world maritime environments, network disruptions caused by sea dynamics, sparse node distribution, and mobility may result in intermittent connectivity. Addressing these network instability issues—such as how to perform robust offloading and caching with intermittent or lossy links—will be a meaningful and practical extension to this work.

References

[1] Z. Wang, B. Lin, Q. Ye, and H. Peng, "Two-tier task offloading for satellite-assisted marine networks: A hybrid Stackelberg-bargaining game approach," *IEEE Internet Things J.*, vol. 12, no. 9, pp. 13047–13060, May 2025.

[2] C. Hu, Y. Pu, F. Yang, R. Zhao, A. Alrawais, and T. Xiang, "Secure and efficient data collection and storage of IoT in smart ocean," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9980–9994, Oct. 2020.

[3] J. Ye, S. Roy, M. Godjevac, and S. Baldi, "A switching control perspective on the offshore construction scenario of heavy-lift vessels," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 1, pp. 470–477, 2020.

[4] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 779–792, Jan. 2021.

[5] H. Zhou, H. Wang, Z. Yu, G. Bin, M. Xiao, and J. Wu, "Federated distributed deep reinforcement learning for recommendation-enabled edge caching," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3640–3656, Nov./Dec. 2024.

[6] M. Dai, C. Dou, Y. Wu, L. Qian, R. Lu, and T. Q. Quek, "Multi-uav aided multi-access edge computing in marine communication networks: A joint system-welfare and energy-efficient design," *IEEE Trans. Commun.*, vol. 72, no. 9, pp. 5517–5531, Sep. 2024.

[7] H. Zhou, M. Li, P. Sun, B. Guo, and Z. Yu, "Accelerating federated learning via parameter selection and pre-synchronization in mobile edge-cloud networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 11, pp. 10313–10328, Nov. 2024.

[8] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE IEEE Conf. Comput. Commun.*, 2018, pp. 207–215.

[9] N. Lin, X. Han, A. Hawbani, Y. Sun, Y. Guan, and L. Zhao, "Deep reinforcement learning based dual-timescale service caching and computation offloading for multi-UAV assisted MEC systems," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 605–617, Feb. 2025.

[10] H. Zeng et al., "USV fleet-assisted collaborative computation offloading for smart maritime services: An energy-efficient design," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 14718–14733, Oct. 2024.

[11] M. Dai, Y. Wu, L. Qian, Z. Su, B. Lin, and N. Chen, "UAV-assisted multi-access computation offloading via hybrid NOMA and FDMA in marine networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 113–127, Jan./Feb. 2023.

[12] Z. Huang et al., "Energy-efficient multi-AAV collaborative reliable storage: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 20913–20926, Jun. 2025.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.

[13] J. Yan, X. Zhang, X. Luo, Y. Wang, C. Chen, and X. Guan, "Asynchronous localization with mobility prediction for underwater acoustic sensor networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2543–2556, Mar. 2018.
[14] T. Lyu, H. Xu, F. Liu, M. Li, L. Li, and Z. Han, "Computing offloading and resource allocation of NOMA based UAV emergency communication in the Marine Internet of Thing," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 15571–15586, May 2024.
[15] Z. Chen, F. Wang, and X. Zhang, "Joint optimization for cooperative service-caching, computation-offloading, and resource-allocations over EH/MEC 6G ultra-dense mobile networks," *IEEE Trans. Wireless Commun.*, vol. 24, no. 7, pp. 5780–5795, Jul. 2025.
[16] Z. Wang, B. Lin, Q. Ye, Y. Fang, and X. Han, "Joint computation offloading and resource allocation for maritime MEC with energy harvesting," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19898–19913, Jun. 2024.
[17] T. Yang, H. Feng, C. Yang, Y. Wang, J. Dong, and M. Xia, "Multivessel computation offloading in maritime mobile edge computing network," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4063–4073, Jun. 2019.
[18] W. Jiang, X. Yuan, C. Huang, and L. Qian, "NOMA-assisted secure computation offloading and resource allocation in Marine Internet of Things," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 1, pp. 534–545, Feb. 2025.
[19] M. Li, L. P. Qian, X. Dong, B. Lin, Y. Wu, and X. Yang, "Secure computation offloading for Marine IoT: An energy-efficient design via cooperative jamming," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6518–6531, May 2023.
[20] T. Yang et al., "Two-stage offloading optimization for energy–latency tradeoff with mobile edge computing in maritime Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5954–5963, Jul. 2020.
[21] H. Zhang, S. Xi, B. Shang, P. Zhang, S. Wu, and C. Jiang, "Energy oriented three-tier computation offloading scheme in maritime edge computing network," *IEEE Trans. Veh. Technol.*, vol. 74, no. 5, pp. 8126–8140, May 2025.
[22] X. Hou, J. Wang, T. Bai, Y. Deng, Y. Ren, and L. Hanzo, "Environment-aware AUV trajectory design and resource management for multi-tier underwater computing," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 474–490, Feb. 2023.
[23] J. Xu, Z. Zhang, J. Wang, Z. Han, and Y. Ren, "Multi-AUV pursuit-evasion game in the Internet of Underwater Things: An efficient training framework via offline reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31273–31286, Oct. 2024.
[24] S. Liu, W. Li, H. Chen, J. Wang, and K. Zhao, "Satellite-assisted task offloading and resource allocation for Ocean of Things edge computing," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 22814–22831, Jul. 2025.
[25] M. Dai, Z. Luo, Y. Wu, L. Qian, B. Lin, and Z. Su, "Incentive oriented two-tier task offloading scheme in marine edge computing networks: A hybrid Stackelberg-auction game approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8603–8619, Dec. 2023.
[26] M. Dai et al., "Latency minimization oriented hybrid offshore and aerial-based multi-access computation offloading for marine communication networks," *IEEE Trans. Commun.*, vol. 71, no. 11, pp. 6482–6498, Nov. 2023.
[27] D. Wang, T. He, Y. Lou, L. Pang, Y. He, and H.-H. Chen, "Double-edge computation offloading for secure integrated space–air–aqua networks," *IEEE Internet Things J.*, vol. 10, no. 17, pp. 15581–15593, Sep. 2023.
[28] J. You, Z. Jia, C. Dong, L. He, Y. Cao, and Q. Wu, "Computation offloading for uncertain marine tasks by cooperation of UAVs and vessels," in *Proc. IEEE Int. Conf. Commun.*, 2023, pp. 666–671.
[29] T. Yang et al., "Multi-armed bandits learning for task offloading in maritime edge intelligence networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4212–4224, Apr. 2022.
[30] H. Li, S. Wu, J. Jiao, X.-H. Lin, N. Zhang, and Q. Zhang, "Energy-efficient task offloading of edge-aided maritime UAV systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1116–1126, Jan. 2023.
[31] M. Stojanovic, "On the relationship between capacity and distance in an underwater acoustic communication channel," *ACM SIGMOBILE Mobile Comput. Commun. Rev.*, vol. 11, no. 4, pp. 34–43, 2007.
[32] D. E. Lucani, M. Stojanovic, and M. Médard, "On the relationship between transmission power and capacity of an underwater acoustic communication channel," in *Proc. IEEE OCEANS 2008-MTS/IEEE Kobe Techno-Ocean*, 2008, pp. 1–6.
[33] J. Guo, S. Song, J. Liu, H. Chen, J.-H. Cui, and G. Han, "A hybrid NOMA-based MAC protocol for underwater acoustic networks," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1187–1200, Apr. 2024.
[34] H. Zhou, Q. Gu, P. Sun, X. Zhou, V. C. Leung, and X. Fan, "Incentive-driven and energy efficient federated learning in mobile edge networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 2, pp. 832–846, Apr. 2025.
[35] L. Liu, L. Cai, L. Ma, and G. Qiao, "Channel state information prediction for adaptive underwater acoustic downlink ofdma system: Deep neural networks based approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9063–9076, Sep. 2021.
[36] Y. Qu et al., "Service provisioning for UAV-enabled mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3287–3305, Nov. 2021.
[37] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter 2017.
[38] D. B. Shmoys and É. Tardos, "An approximation algorithm for the generalized assignment problem," *Math. Program.*, vol. 62, no. 1, pp. 461–474, 1993.
[39] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24611–24624.
[40] Y. Zhang, Z. Na, Z. Wen, A. Nallanathan, and W. Lu, "Joint service caching, computation offloading and resource allocation for dual-layer aerial Internet of Things," *Comput. Netw.*, vol. 257, 2025, Art. no. 110974.
[41] X. Gao and L. Zhai, "Service experience oriented cooperative computing in cache-enabled UAVs assisted MEC networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9721–9736, Oct. 2024.
[42] J. Huang, M. Zhang, J. Wan, Y. Chen, and N. Zhang, "Joint data caching and computation offloading in UAV-assisted Internet of Vehicles via federated deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 17644–17656, Nov. 2024.
[43] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Trans. Serv. Comput.*, vol. 17, no. 4, pp. 1548–1564, Jul./Aug. 2024.



Zhaoxiang Huang received the BEng degree in software engineering from Liaoning University, Shenyang, China, in 2021. He is currently working toward the PhD degree with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China. His research interests include mobile crowdsensing, edge computing, and edge storage.



Zhiwen Yu (Senior Member, IEEE) received the PhD degree in computer science from Northwestern Polytechnical University, Xi'an, China, in 2005. He is currently the vice president of Harbin Engineering University, Harbin, China, and a professor with Northwestern Polytechnical University. He was an Alexander Von Humboldt fellow with Mannheim University, Germany, and a research fellow with Kyoto University, Kyoto, Japan. His research interests include ubiquitous computing, mobile crowd sensing, and human computer interaction.



Liang Wang (Member, IEEE) received the PhD degree from the Shenyang Institute of Automation (SIA), Chinese Academy of Sciences, Shenyang, China, in 2014. He is currently a professor with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China. His research interests include ubiquitous computing, mobile crowd sensing, and crowd computing.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.



Yingnan Zhao (Member, IEEE) received the PhD degree in computer science from the Harbin Institute of Technology, Harbin, China, in 2023. He is currently an associate professor in the School of Computer Science and Technology, Harbin Engineering University, China. His research interests include deep reinforcement learning and mobile edge computing.



Huan Zhou (Senior Member, IEEE) received the PhD degree from the Department of Control Science and Engineering, Zhejiang University. He was a visiting scholar with the Temple University from November 2012 to May, 2013, and a CSC supported postdoc fellow with the University of British Columbia from November 2016 to November 2017. He is currently a professor with Northwestern Polytechnical University, Xi'an, China. He was a lead guest editor of the *Pervasive and Mobile Computing*, and Special Session chair of the 3 rd International Conference on Internet of Vehicles (IoV 2016), and TPC member of IEEE WCSP'13'14, CCNC'14'15, ICNC'14'15, ANT'15'16, IEEE Globecom'17'18, ICC'18'19, etc. He has published more than 50 research papers in some international journals and conferences, including the *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Vehicular Technology* and so on. His research interests include mobile social networks, vehicular ad hoc networks, opportunistic mobile networks, and mobile data offloading. He received the Best Paper Award of I-SPAN 2014 and I-SPAN 2018, and is currently serving as an associate editor of the *IEEE Access* and *EURASIP Journal on Wireless Communications and Networking*.



Bin Guo (Senior Member, IEEE) received the PhD degree in computer science from Keio University, Minato, Japan, in 2009. He is currently a professor with Northwestern Polytechnical University, Xi'an, China. He was a postdoctoral Researcher with the Institut TELECOM SudParis, Essonne, France. His research interests include ubiquitous computing, mobile crowd sensing, and HCI.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:42:07 UTC from IEEE Xplore. Restrictions apply.